



Universidad Nacional Autónoma de México
Facultad de Ingeniería



Práctica 5

Silverio Martínez Andrés

Lab. Organización y Arquitectura de
Computadoras

Grupo: 8

Semestre 2026-2

Profesor: Jorge Ferat Toscano

Fecha de entrega: 19 de abril de 2026

Práctica 5

Objetivo

Generar de diferentes formas un multiplexor (MUX) por medio del lenguaje VHDL utilizando el software Quartus II

Introducción

En el ecosistema del diseño digital, el multiplexor (MUX) se posiciona como un bloque funcional crítico. Su capacidad para conmutar entre diversas señales de entrada y dirigir una única salida —basándose en señales de control— es el pilar de la gestión de flujo de datos en arquitecturas complejas, desde sistemas de comunicación hasta procesadores de alto rendimiento.

Esta práctica se centra en el diseño, implementación y simulación de un multiplexor 2:1 empleando el lenguaje de descripción de hardware VHDL. Para profundizar en la versatilidad del lenguaje, el desarrollo aborda cuatro metodologías distintas:

- **Lógica de compuertas:** Descripción estructural básica.
- **Señales intermedias:** Organización de nodos internos.
- **Sentencias concurrentes:** Flujo de datos directo.
- **Procesos secuenciales:** Modelado conductual.

El objetivo central es contrastar estas variantes de codificación y verificar que, pese a las diferencias sintácticas, el comportamiento lógico permanece invariante. La validación se realiza mediante herramientas de grado industrial como Quartus Prime (Waveform Editor) y ModelSim-Altera, consolidando competencias esenciales en síntesis de hardware y verificación funcional para la ingeniería moderna.

Desarrollo

1) Crear un proyecto con nombre practica 5, utilizando la misma forma de generarlo conforme a lo realizado en la anterior practica 1

Paso 1: Seleccionamos la opción de “New Project Wizard”

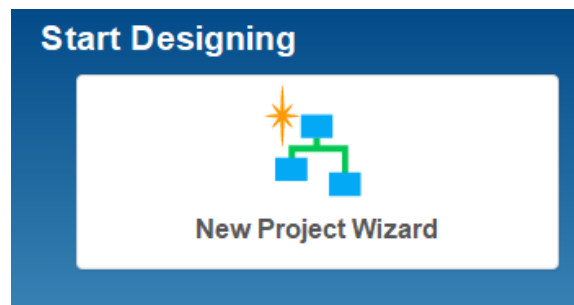


Imagen 1: New Project Wizard

Paso 2: Seleccionamos la carpeta donde queremos guardar el proyecto

 New Project Wizard

Directory, Name, Top-Level Entity

What is the working directory for this project?

C:/Users/imano/Documents/Lab. Arquí/Practica 5

Imagen 2: Directory for this project

Paso 3: Le asignamos el nombre del proyecto

What is the name of this project?

Practica5

What is the name of the top-level design entity for this project? This name is case sensitive and must exactly match the entity name in the design file.

Practica5

Use Existing Project Settings...

Imagen 3: Name of this Practica3

Paso 4:

 New Project Wizard

×

Project Type

Select the type of project to create.

☒ Empty project

Create new project by specifying project files and libraries, target device family and device, and EDA tool settings.

Imagen 4: Project Type

Paso 5: Añadimos el archivo “Practica3”

Add Files

Select the design files you want to include in the project. Click Add All to add all design files in the project directory to the project.

Note: you can always add design files to the project later.

File name: Practica5

Add

Imagen 5: Add Files

Paso 6: Llegamos a esta ventana y ponemos en auto

New Project Wizard

Family & Device Settings

Select the family and device you want to target for compilation.
You can install additional device support with the Install Devices command on the Tools menu.

To determine the version of the Quartus Prime software in which your target device is supported, refer to the [Device Support List](#) webpage.

Device family

Family: Cyclone IV E

Devices: All

Target device

☒ Auto device selected by the Filter

☐ Specific device selected in 'Available devices' list

☐ Other: n/a

Show in 'Available devices' list

Package: Any

Pin count: Any

Core Speed grade: Any

Name filter:

☒ Show advanced devices

Available devices:

Name	Core Voltage	LEs	Total I/Os	GPIOs	Memory Bits	Embedded multiplier 9-bit elements
EP4CE6E22A7	1.2V	6272	92	92	276480	30
EP4CE6E22C6	1.2V	6272	92	92	276480	30
EP4CE6E22C7	1.2V	6272	92	92	276480	30
EP4CE6E22C8	1.2V	6272	92	92	276480	30
EP4CE6E22C8L	1.0V	6272	92	92	276480	30
EP4CE6E22C9L	1.0V	6272	92	92	276480	30
EP4CE6E22I7	1.2V	6272	92	92	276480	30
EP4CE6E22I8L	1.0V	6272	92	92	276480	30

< >

< Back Next > Finish Cancel Help

Imagen 6: Family & Device Settings

Paso 7: Para terminar, nos aparecerá la siguiente ventana y le daremos clic en “Finish”

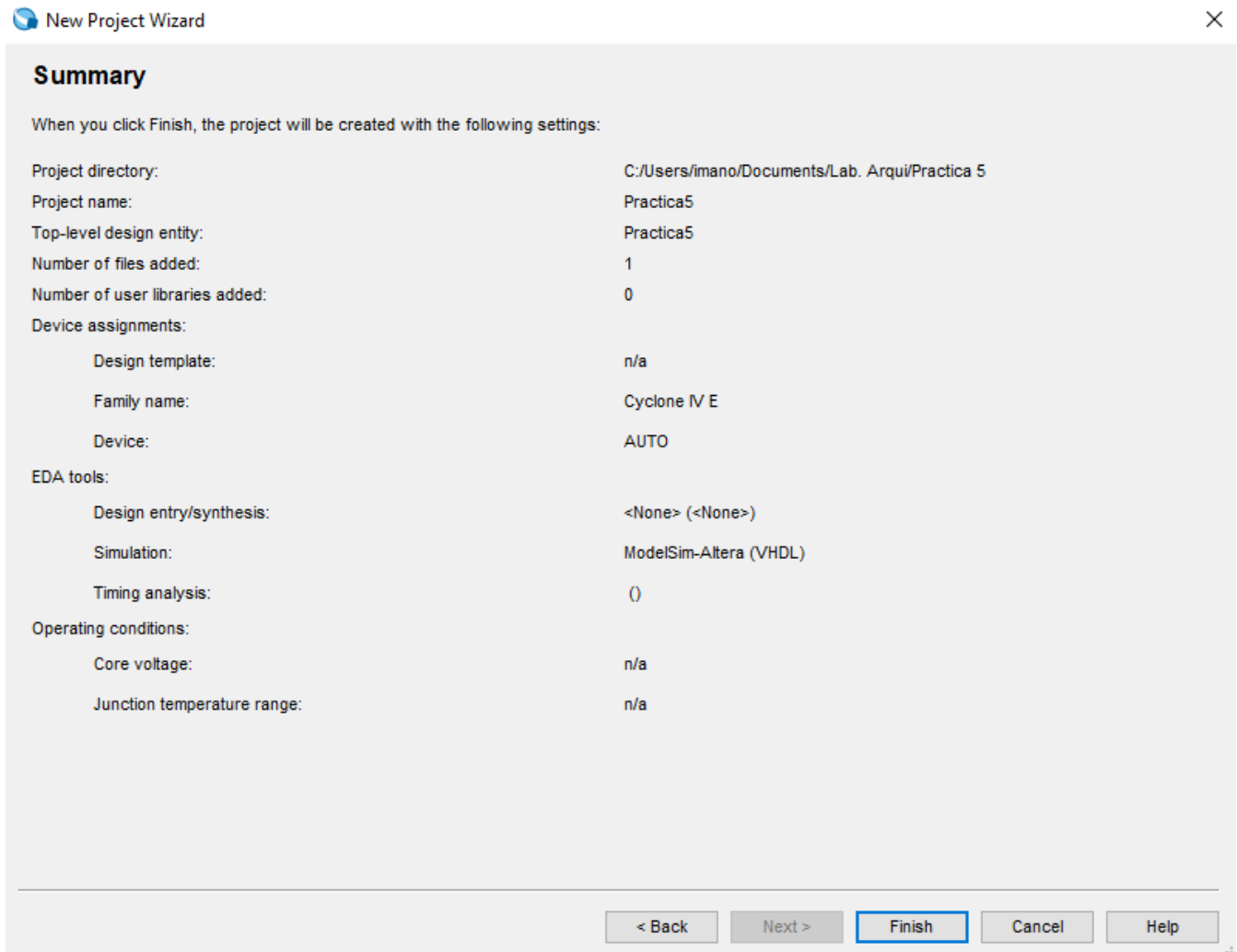


Imagen 7: Summary

2) El tipo de dispositivo a utilizar considere Auto

En la siguiente imagen vemos, que pusimos Auto en la configuración:

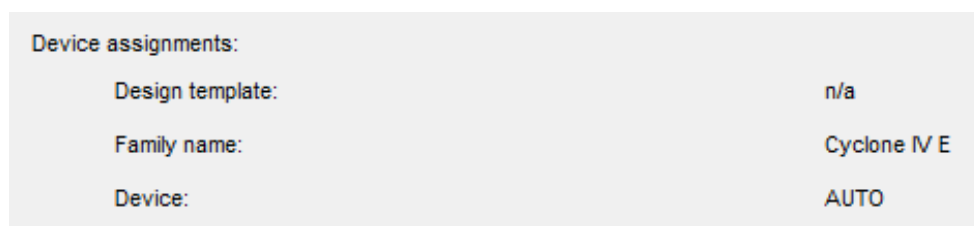


Imagen 8: Device AUTO

3) Crear un archivo nuevo de diseño de tipo VHDL

Paso 1: Damos clic en “File” y después en “New”

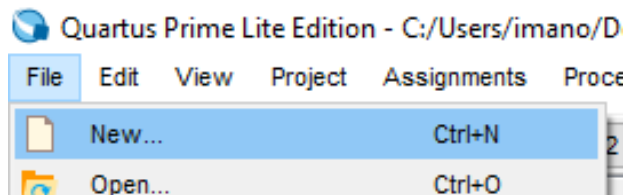


Imagen 9: Nuevo archivo

Paso 2: En la siguiente ventana, dar clic en “VHDL File” y después clic en “Ok”

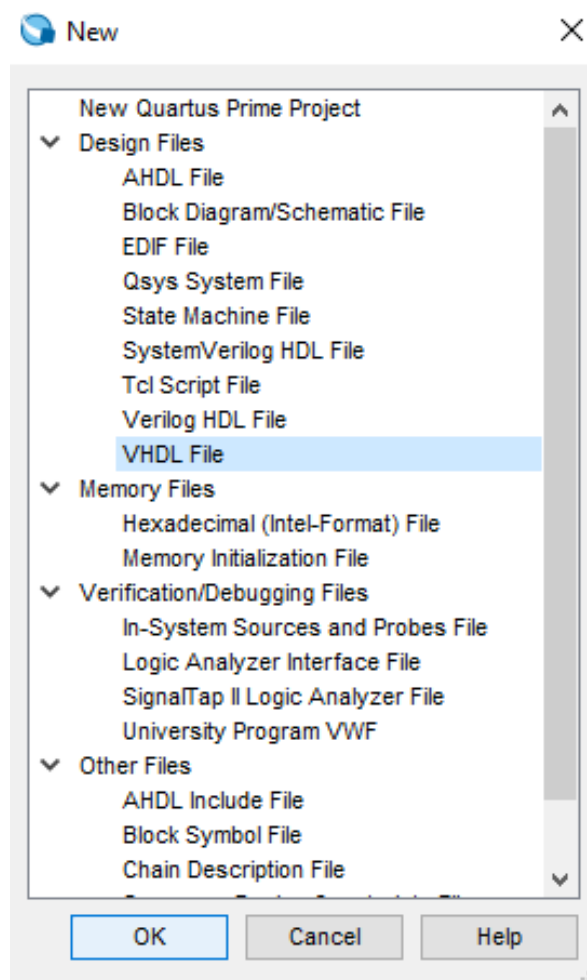


Imagen 10: VHDL File

4) Una vez que este generado teclear en el editor el código

Opción 1: Mediante puertas

```
--Mux varias formas
--Opción mediante compuertas
library ieee;
use ieee.std_logic_1164.all;
entity Practica5 is
port (
    pto0: in std_logic;
    pto1: in std_logic;
    sel : in std_logic;
    ptops: out std_logic);
end Practica5;

--Opción mediante compuertas
architecture multiplexor of Practica5 is
begin
    ptops<= (pto0 and (not sel)) or (pto1 and sel);
end multiplexor;
```

Imagen 11: Código

Opción 2: Opción mediante compuertas. Salidas intermedias

```
--Mux varias formas
--Opción mediante compuertas
library ieee;
use ieee.std_logic_1164.all;
entity Practica5 is
port (
    pto0: in std_logic;
    pto1: in std_logic;
    sel : in std_logic;
    ptops: out std_logic);
end Practica5;

-- Opción mediante compuertas
-- Salidas intermedias
architecture multiplexor of Practica5 is
    signal sig0, sig1: std_logic;
begin
    sig0 <= pto0 and (not sel);
    sig1 <= pto1 and sel;
    ptops <= sig0 or sig1;
end multiplexor;
```

Imagen 12: Código

Opción 3: Opción mediante sentencia concurrente, utilizando when

```
--Mux varias formas
--Opción mediante compuertas
library ieee;
use ieee.std_logic_1164.all;
entity Practica5 is
port (
    pto0: in std_logic;
    pto1: in std_logic;
    sel : in std_logic;
    ptops: out std_logic);
end Practica5;

architecture multiplexor of practica5 is
begin
    ptops <= pto0 when sel = '0' else pto1;
end multiplexor;
```

Imagen 13: Código

Opción 4: Opción mediante proceso

```
--Mux varias formas
--Opción mediante compuertas
library ieee;
use ieee.std_logic_1164.all;
entity Practica5 is
port (
    pto0: in std_logic;
    pto1: in std_logic;
    sel : in std_logic;
    ptops: out std_logic);
end Practica5;

-- Opción mediante sentencias concurrente
-- Opción mediante proceso
architecture multiplexor of practica5 is
begin
    proceso: process (sel, pto0, pto1)
    begin
        if sel = '0' then
            ptops <= pto0;
        else
            ptops <= pto1;
        end if;
    end process;
end multiplexor;
```

Imagen 14: Código

Análisis del código

El código implementa un **multiplexor 2:1** (MUX 2 a 1).

- Entradas:
 - pto0 (bit de entrada 0)
 - pto1 (bit de entrada 1)
 - sel (señal de selección)
- Salida:
 - ptops (la salida seleccionada).

El MUX funciona así:

- Si sel = '0', la salida es pto0.
- Si sel = '1', la salida es pto1.

Tienes varias formas de implementar el mismo MUX en VHDL:

1. Mediante compuertas lógicas.
2. Mediante compuertas con señales intermedias.
3. Mediante sentencia concurrente when ... else.
4. Mediante un proceso secuencial (process).

5) Compilar el programa Revisar que la compilación fue de manera exitosa

Como vemos en la siguiente imagen, se muestra que fue exitosa la compilación:

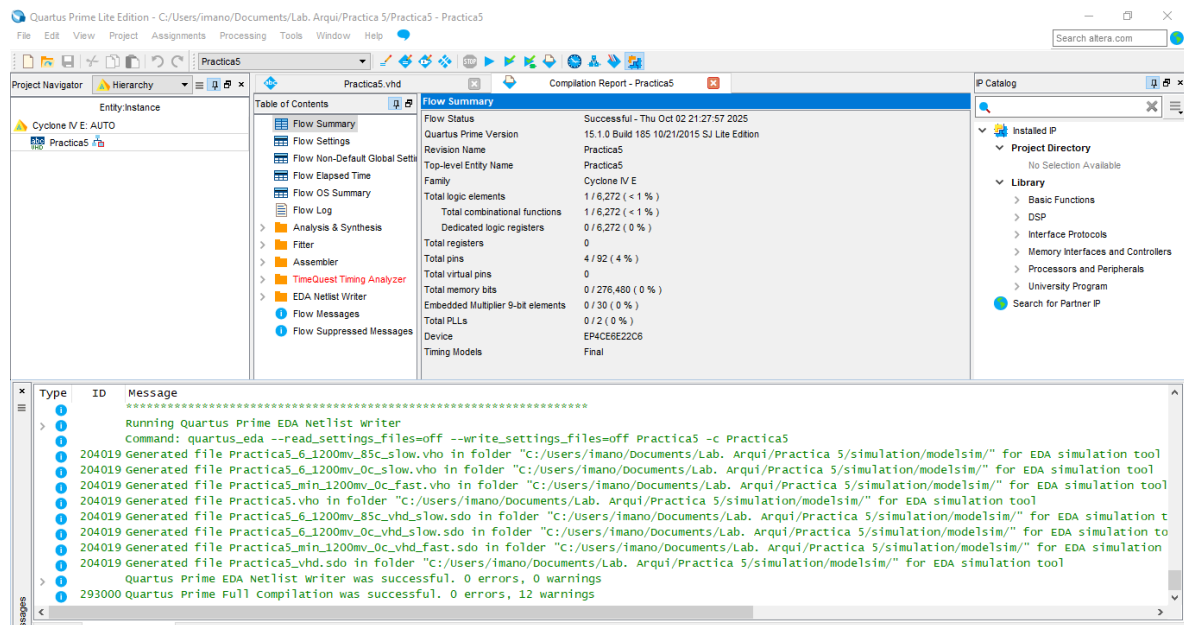


Imagen 15: Compilación

6) Procedemos a revisar el circuito sintetizado (RTL Viewer) Se observará en la pantalla el diagrama correspondiente al circuito

Paso 1: Nos dirigimos en la pestaña de “Tools”, buscamos “Netlist Viewers” y finalmente damos clic en “RTL Viewer”

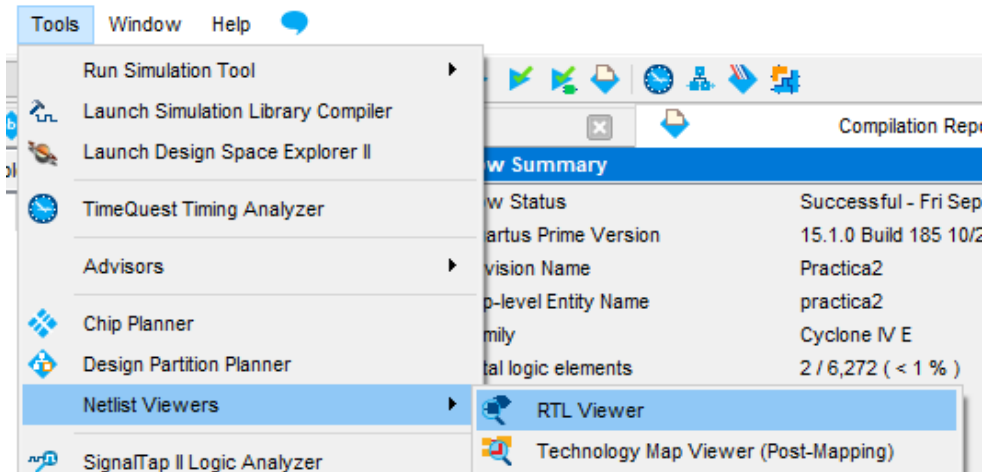


Imagen 16: RTL Viewer

Opción 1: Mediante puertas. Y **Opción 2:** Opción mediante compuertas. Salidas intermedias

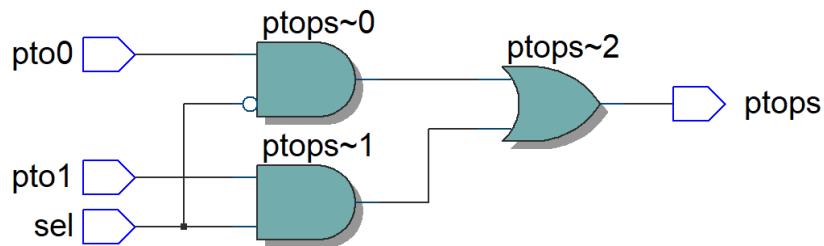


Imagen 17: Diagrama

Opción 3: Mediante sentencia concurrente, utilizando when. Y **Opción 4:** Mediante proceso.

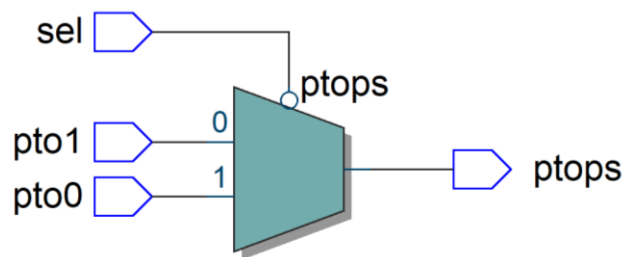


Imagen 18: Diagrama

7) Una compilación exitosa no garantiza un funcionamiento satisfactorio, pues podrían existir errores de lógica. Por lo anterior es conveniente simular comportamiento del circuito. Para ello se procederá a realizar la simulación de este

Simulación 1: Como quiero hacer la simulación en Waveform Editor, entonces seguimos el siguiente paso. En la ventana New, en la sección Verification/Debugging Files seleccionar la opción University Program VWF.

Opción 1: Mediante puertas

Ejemplo: Pto0 = 1, Pto1=0, Sel=0, Ptops=1

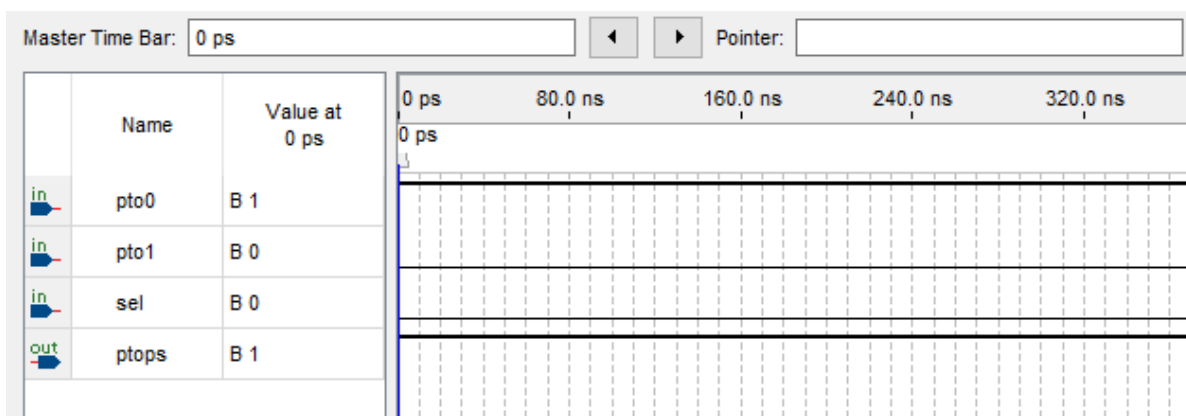


Imagen 19: University Program VWF

Cuando la señal de selección (sel) vale 0, el multiplexor elige la entrada **Pto0**, por lo tanto la salida (Ptops) toma su valor (1).

Opción 2: Opción mediante compuertas. Salidas intermedias

Ejemplo: Pto1 = 0, Pto1=0, Sel=1, Ptops=0

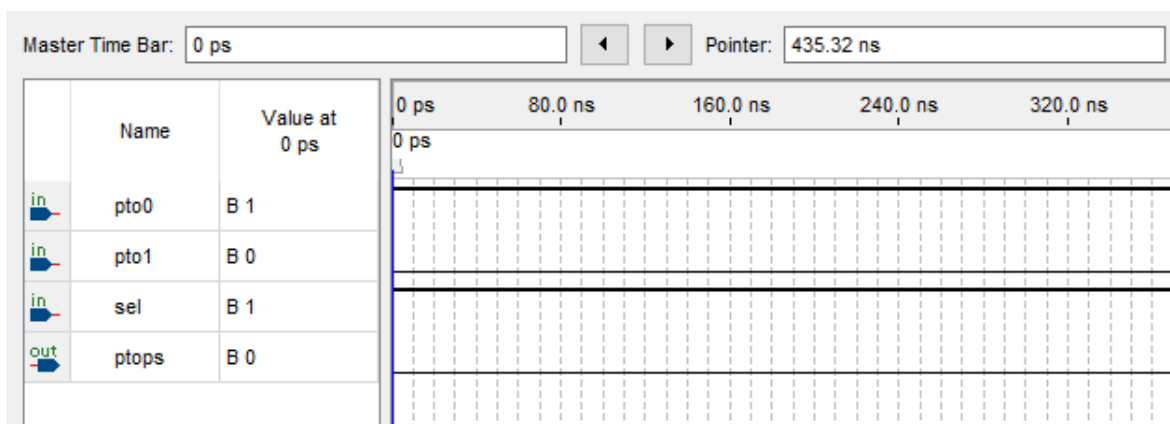


Imagen 20: University Program VWF

La señal de selección vale 1, así que el multiplexor toma la entrada **Pto1**. Como Pto1 = 0, la salida también será 0.

Opción 3: Opción mediante sentencia concurrente, utilizando when

Ejemplo: Pto0 = 0, Pto1=1, Sel=1, Ptops=1

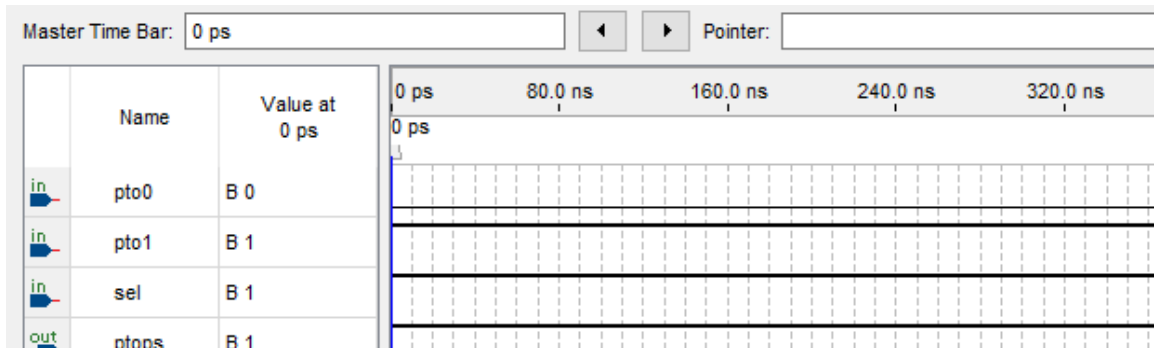


Imagen 21: University Program VWF

Aquí se usa una descripción concurrente en VHDL con la sentencia `when`. Al tener `sel = 1`, la salida toma el valor de **Pto1 = 1**.

Opción 4: Opción mediante proceso

Ejemplo: Pto0 = 0, Pto1=0, Sel=1, Ptops=0

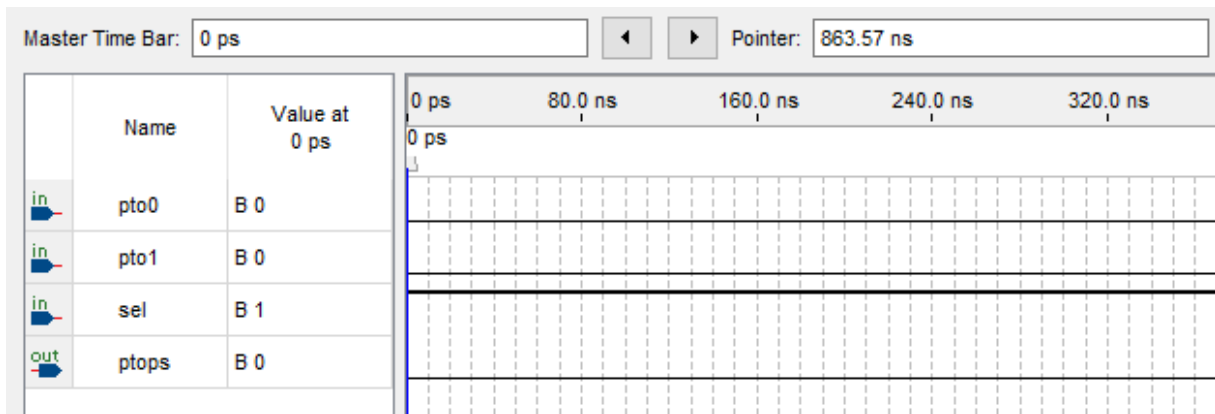


Imagen 22: University Program VWF

Dentro del proceso secuencial en VHDL, si `sel = '1'` se asigna `Pto1`, que vale 0; por eso la salida también es 0.

Simulación 2

Utilizamos la otra simulación que es la misma interpretación

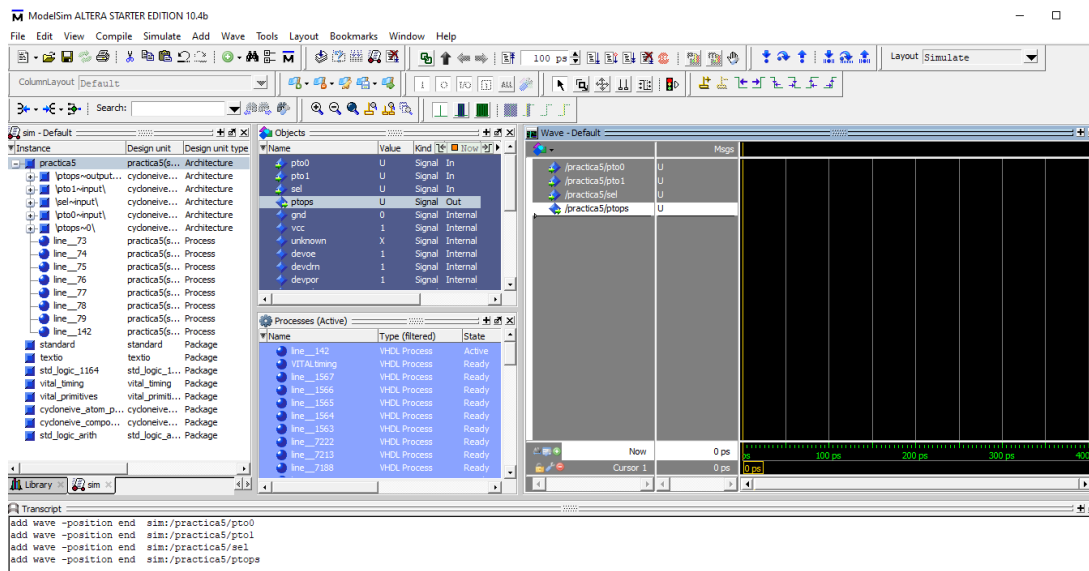


Imagen 23: ModelSim – Altera.

Opción 1: Mediante puertas

Ejemplo: Pto1 = 0, Pto1=0, Sel=0, Ptops=1

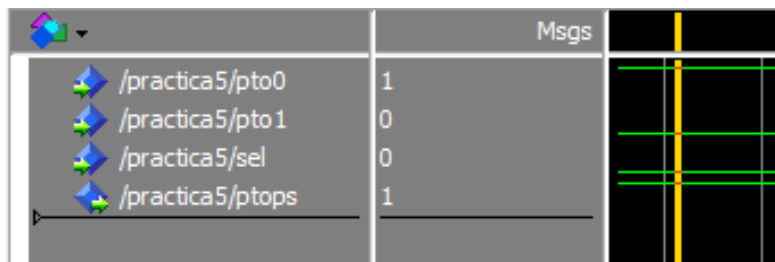


Imagen 24: ModelSim – Altera.

El multiplexor selecciona la entrada **Pto0** (porque Sel=0), y la salida refleja ese valor.

Opción 2: Opción mediante compuertas. Salidas intermedias

Ejemplo: Pto0 = 1, Pto1=0, Sel=1, Ptops=0

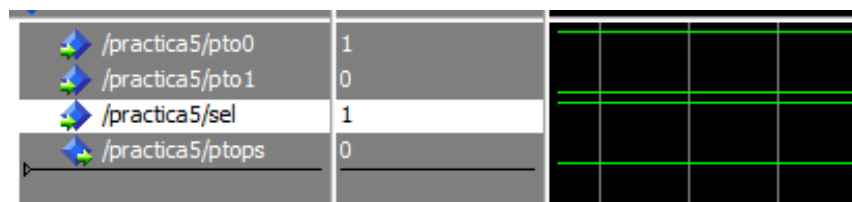
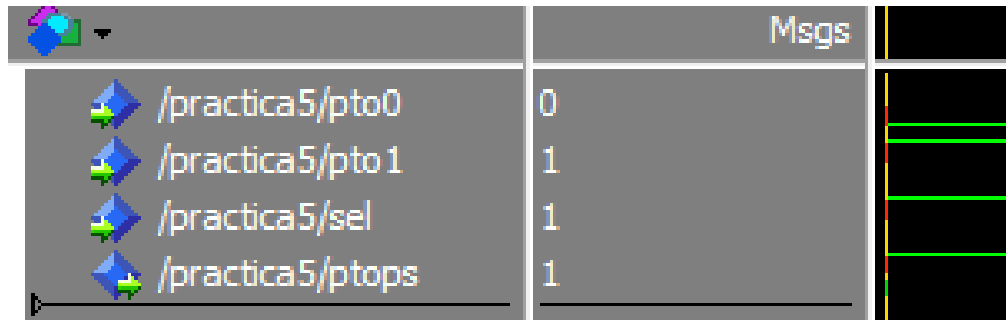


Imagen 25: ModelSim – Altera.

El selector vale 1, por lo que el sistema toma el valor de **Pto1**, que en este caso es 0.

Opción 3: Opción mediante sentencia concurrente, utilizando when

Ejemplo: Pto0 = 0, Pto1=1, Sel=1, Ptops=1



The image shows a ModelSim wave window titled 'Wave - Default'. It displays four signals: /practica5/pto0, /practica5/pto1, /practica5/sel, and /practica5/ptops. The values for these signals are 0, 1, 1, and 1 respectively. The waveforms are shown as horizontal lines on a black background, with green lines for the signals and a yellow line for the time axis.

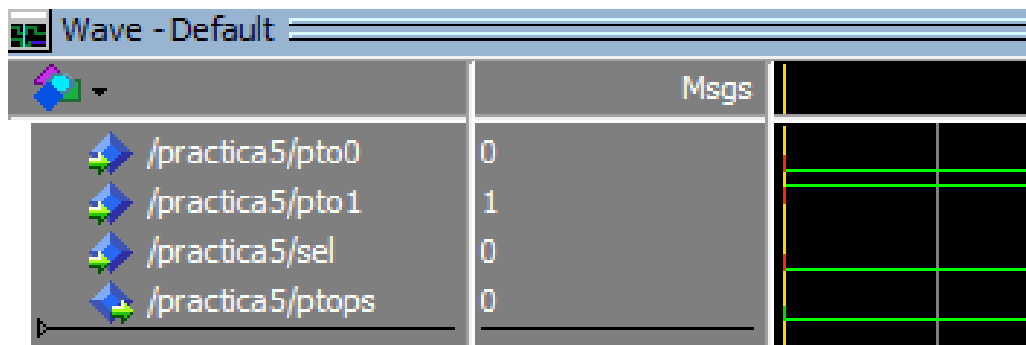
Signal	Value
/practica5/pto0	0
/practica5/pto1	1
/practica5/sel	1
/practica5/ptops	1

Imagen 26: ModelSim – Altera.

En esta descripción, cuando $sel = 1$, la salida adopta el valor de la entrada **Pto1** = 1. Esto muestra la correcta operación del multiplexor.

Opción 4: Opción mediante proceso

Ejemplo: Pto0 = 0, Pto1=1, Sel=0, Ptops=0



The image shows a ModelSim wave window titled 'Wave - Default'. It displays four signals: /practica5/pto0, /practica5/pto1, /practica5/sel, and /practica5/ptops. The values for these signals are 0, 1, 0, and 0 respectively. The waveforms are shown as horizontal lines on a black background, with green lines for the signals and a yellow line for the time axis.

Signal	Value
/practica5/pto0	0
/practica5/pto1	1
/practica5/sel	0
/practica5/ptops	0

Imagen 27: ModelSim – Altera.

Como $sel = 0$, el multiplexor selecciona **Pto0**, que vale 0. Por eso la salida permanece en bajo (0).

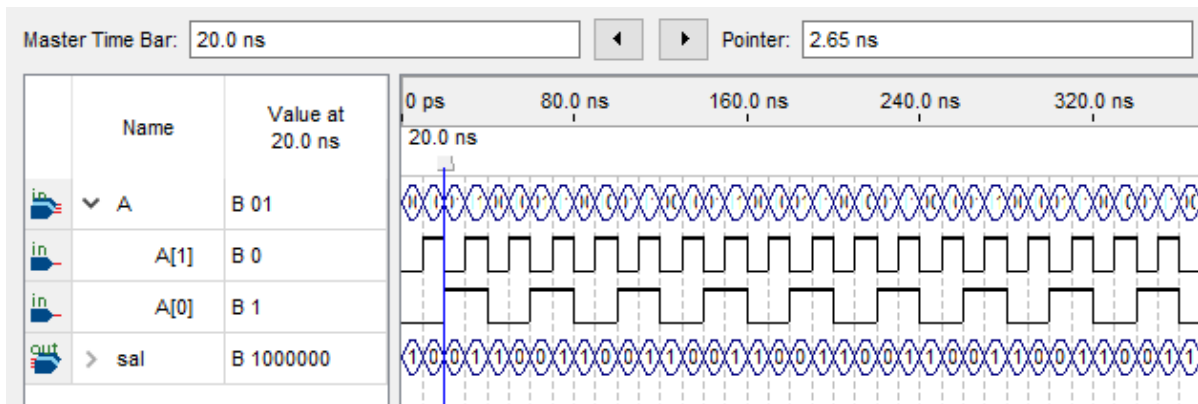


Imagen 34: Simulación

Dado que:

- A_0 cambia cada **10 ns** (porque su periodo es 20 ns, y alterna entre 0 y 1 cada mitad),
- A_1 cambia cada **20 ns** (porque su periodo es 40 ns),

se obtiene la siguiente secuencia de combinaciones binarias en el tiempo:

Tiempo (ns)	A_1	A_0	Valor de A (A_1A_0)
0–10	0	0	"00"
10–20	0	1	"01"
20–30	1	0	"10"
30–40	1	1	"11"
40–50	0	0	"00"
(y se repite)			

Así, cada **40 ns** se completa un ciclo completo de los 4 valores posibles de A.

Tu proceso usa un **case** sobre A, así que sal tomará distintos valores según esa tabla.

A (A_1A_0)	Valor de sal
"00"	"0001001"
"01"	"1000000"
"10"	"1000111"
"11"	"0001000"

SIMULACIÓN 2

Utilizamos la otra simulación que es la misma interpretación

0–10	"00"	"0001001"
10–20	"01"	"1000000"
20–30	"10"	"1000111"
30–40	"11"	"0001000"
40–50	"00"	"0001001"
<i>(y se repite cada 40 ns)</i>		

Lo que sucede:

- La entrada A cambia automáticamente con el tiempo debido a los periodos distintos de A_0 y A_1 .
- La salida sal responde instantáneamente a esos cambios, tomando los valores definidos en el case.
- Se genera una **secuencia cíclica de 4 estados** que se repite cada **40 ns**.

Visualmente (en una simulación o display):

- Verías que sal cambia cada 10 ns, recorriendo los cuatro patrones predefinidos.
- Si sal controla un display de 7 segmentos, éste mostraría **4 figuras o números distintos** que se van alternando rápidamente (cada 10 ns).

Conclusión

A lo largo de esta práctica fue posible asimilar e implementar el comportamiento de un multiplexor 2:1 (MUX) explorando diversas metodologías de descripción en VHDL: compuertas lógicas, señales intermedias, sentencias concurrentes (when...else) y procesos secuenciales (process). El análisis comparativo de estas cuatro aproximaciones demostró que, si bien todas convergen en el mismo resultado lógico, cada una presenta una organización y un nivel de abstracción distintos dentro del diseño digital.

La etapa de compilación, complementada con la inspección del RTL Viewer, facilitó la visualización de la arquitectura interna del circuito y confirmó que la síntesis se llevó a cabo de manera exitosa y consistente con el código escrito. A partir de esto, quedó en evidencia el papel fundamental que desempeña la simulación funcional, pues es el único medio que permite validar verdaderamente el comportamiento del diseño frente a las distintas combinaciones posibles de entradas.

Para concluir, las simulaciones ejecutadas tanto en el Waveform Editor de Quartus como en ModelSim–Altera verificaron el funcionamiento esperado del multiplexor: con la señal de selección Sel en 0, la salida refleja el valor de Pto0, mientras que con Sel en 1, reproduce el de Pto1. Este ejercicio fortaleció la comprensión sobre el flujo de datos gobernado por señales de selección, al tiempo que consolidó competencias en la descripción, compilación y verificación de circuitos digitales mediante VHDL.

Referencias

1. Intel. (2024). *Using the Quartus Prime software for FPGA design*. Recuperado el 11 de octubre de 2025, de <https://www.intel.com/content/www/us/en/software/programmable/quartus-prime/overview.html>
2. FPGA4Student. (2023). *VHDL tutorial: Multiplexer design using VHDL*. Recuperado el 11 de octubre de 2025, de <https://www.fpga4student.com/2017/06/vhdl-code-for-2-to-1-multiplexer.html>
3. AllAboutCircuits. (2024). *Introduction to multiplexers and demultiplexers*. Recuperado el 11 de octubre de 2025, de <https://www.allaboutcircuits.com/technical-articles/introduction-to-multiplexers-and-demultiplexers/>